Statistical Methods 2024: Assignment 2 – Radial Profile of Gaia Clusters

This is the second of two assignments, for which extensive help is available during the tutorials. It is worth 25% of the final course grade.
The deadline for this assignment is Tuesday 27 Jan at 12:00 noon. Late work which is submitted up to 24 hours after the deadline will receive a penalty of -20% of the awarded grade, while work submitted 24-48 hours after the deadline will receive a penalty of -40% of the awarded grade. Work submitted later than this will not be assessed. The rubric for the assessment of all the assignments, listing the categories assessed and the requirements for each of them, will be provided separately on Canvas.

What you should submit
You should submit your work via Canvas. ***It must be in the form of a Jupyter notebook.*** Make sure that you upload the correct file, and check that all the cells run successfully (***and in the correct order***, from start to finish) before you submit! Advice: before you submit, restart your kernel and run through all the cells in order, to make sure they run and produce the expected output. Questions that ask you to write text should be submitted as text cells within that same notebook.
Before you start it is essential that you read through the assignment grading explanation document on Canvas, since this explains what we expect from you in your answers. When answering each question, use markdown cells for explanations, assumptions and comments on your results: do not include these as comments in code cells, which are reserved only for comments about the code itself!
Some questions ask you to make a prediction about what you will see in the data before you look at it: **there will be no points deducted for the prediction being incorrect**! You can achieve full points with an incorrect prediction, and task completion and quality will be judged based on whether you applied the statistical reasoning you are learning in the class to motivate your prediction.

Remember that the usual plagiarism rules apply to your work: if you cut-and-paste code from somewhere/someone else (including code generated by an AI) you must cite the source (simply replacing variable names is not sufficient to make it your own!). See the grading explanation for more details. We also expect you to help each other, at least early on, and/or be inspired by methods you see online, so programming ***your own version*** (i.e. not cut and pasted) of someone else's method is fine and does not require citation.
You may use AI for this work in the following ways:

- To help you **implement code**: You may use AI to figure out the precise syntax for rote programming tasks, such as plotting and database manipulation, but **do not ask for and copy full solutions to assignment questions**
- to help you **debug code**: you may ask the AI for hints, explanations, or checks, to help you think through error messages, but **do not ask for or simply copy full solutions**
- to **help you understand concepts (e.g. for astrophysical quantities and processes you may not know) and think through mistakes**, but AI **cannot and must not decide for you** which assumptions are reasonable, which tests are appropriate, or how to interpret results.

You may **not** ask AI to generate full functions for you, or to provide full answers to the questions. **To do so is considered academic fraud and will be addressed as such**. See the course guide on Canvas for more details.
Note that you are responsible for the correctness of all work you submit, no matter its source. You must be able to explain your solution in detail if asked. You must flag any support by AI clearly and specifically (e.g. if it helped you figure out code, make sure you specify how in a docstring or text cell).
Any written answer must explicitly reference at least one figure, table, or numerical value produced by *your own code*. Be sure that you describe what features in the figure you are using inform your interpretation! Answers that could plausibly have been written *before running the notebook* will receive no credit.

The Assignment:

Why do we care about radial density profiles?
Clusters are groups of stars that formed together and remain (at least partially) gravitationally associated. Their **radial surface-density profile**—how the projected number density of stars changes with distance from the cluster centre—provides a compact summary of structure. In practice, such profiles are used to (i) measure a characteristic core scale, (ii) quantify how concentrated the cluster is, (iii) separate the cluster from the background field population, and (iv) compare clusters in a consistent way to study evolution and disruption. In this question you will treat the radial profile as a **function-fitting problem**

and focus on Bayesian inference, sampling, and model checking. You will analyse one cluster from list in `assignment2_clusters.dat` using sky positions (RA/Dec). You will use a King-like radial profile [(King, 1962)](King, 1962) defined as

$$\Sigma_{\rm cl}(R) = \Sigma_0\left(\frac{1}{\sqrt{1+(R/r_c)^2}} - \frac{1}{\sqrt{1+(r_t/r_c)^2}}\right)^2, \quad 0 \le R \le r_t.$$

with parameters $\Sigma_0 > 0$ the central surface density, $r_c > 0$ the core radius, and $r_t > r_c$ the tidal radius. The core radius is defined as the distance at which the apparent surface luminosity has dropped by half (usually of the order of a few pc). The tidal radius is the distance from the center of the globular cluster at which the external gravitation of the galaxy has more influence over the stars in the cluster than does the cluster itself. This is the distance at which the individual stars belonging to a cluster can unbound by the galaxy, and is usually much larger than the core radius (can be over 100pc or, depending on distance, tens of arcmin).
Your goal is to infer the physical properties of the cluster, encoded as parameters of the model.

General hints:

- There are statistical libraries that implement a lot of different useful functions, statistical tests and methods that will be helpful here, you are not expected to programme them all from scratch! Two common examples used in statistical computation are `scipy.stats` and `statsmodels`. You are encouraged to use the tools available to you in open-source software packages, as we do when we do scientific analyses..
- I have provided for you a Python file with some functions that should help you in a couple of places. Before you start writing code, check whether you can find what you need there.
- Most of the analysis you'll have to do more than once. If you make your code modular and reusable (e.g. by using functions judiciously), you'll have a much faster and easier time re-running where you need to without having to copy-paste and adjust code.
- That said, while it is possible to write general functions for the log-likelihood and the log-prior, it's kind of a hassle and takes a long time and hard thinking to make it work. For the purpose of this exercise, you are allowed to hard-code values (e.g. for the hyperparameters of the prior distribution) and details of the specific model (e.g. the King model) into the log-likelihood/log-prior functions you write and duplicate/adapt functions where necessary to make your lives easier.
- If you're worried about bugs, always test out your functions! Your log-prior function is relatively easily tested with values that you know are either possible or impossible according to the prior. Try out a few to make sure you've defined everything correctly. For the likelihood, if you're concerned, here's something you can do: (1) pick a set of parameters; (2) simulate a density profile and turn into expected counts using the area; (3) draw from a Poisson distribution to generate simulated data. (4) Fix all parameters but one to the true value, and compute the likelihood for an array of values of the dimension you haven't fixed. (5) plot the likelihood versus parameter value, and compare to the true value you used to simulate the data, the likelihood should peak near the true value.
- If you're using emcee, there is a nice step-by-step tutorial for setting up your inference
- If your code is slow, see if the variable you're computing with is a `pandas.DataFrame` or `pandas.Series`. Sometimes, you can gain considerable speedup by turning a `pandas.Series` into a numpy.ndarray using `myseries.values`. MCMC with emcee on my computer takes about ~1 minute for ~5000 samples.

Exercise 1 – Initial Setup and computing the cluster centre and radii (5 pts)

(a) First, load in the stars data as shown in Assignment 1. Then load the list of clusters in `assignment2_clusters.dat` into a pandas `Series` object. To assign a cluster, write down the last four digits of your StudentID k, and compute your unique cluster ID using those numbers and the length of the list you loaded from `assignment2_clusters.dat`:

```
cluster_index = k % len(assigment2_clusters)
cluster_name = assigment2_clusters.iloc[cluster_index]
```

Here, the % symbol computes the modulo (remainder after dividing k by the length of the cluster dataframe)**. In your notebook, be sure you note down k, the cluster index and the name of the cluster you will be working with.**

(b) Use the functions provided in the helper Python script to compute a cluster centre on the sphere and angular separations for each star from that centre. From there, you can compute the projected radius in arcmin for each star:

$$R_i = 60 \times \theta_i \times \frac{180}{\pi}, \quad \theta_i = \text{angular\_separation}(\cdot).$$

(i) Make a scatter plot of RA vs Dec with the centre marked.

(ii) Comment on any structure you see in the scatter plot and how this might affect a radially binned profile.

(c) We will now bin the stellar radii into a radial density profile by gathering all stars in annuli (donut-shaped area between two bounds R_j and R_{j+1}). First, **predict**: How will the number of low-count bins change when going from $J = 15$ to $J = 50$ bins? Give a specific numeric expectation (e.g. "I expect the number of bins with counts $N < 5$ to increase by X amount") and justify your prediction using the scatter plot you've made.

(d) Set the maximum radius R_{max} to the largest angular projected radius, and define bin edges $0 = R_0 < R_1 < \cdots < R_J = R_{max}$. Make radial profiles for both $J = 15$ and $J = 50$. For each profile, compute:

- star counts N_j for each annulus j
- the annulus area $A_j = \pi(R_{j+1}^2 - R_j^2)$ (arcmin^2);
- mid-radius $R_{j,mid} = (R_j + R_{j+1})/2$;
- and empirical surface density $\hat{\Sigma}_j = \frac{N_j}{A_j}$
- how many bins satisfy $N_j < 1$, $N_j < 5$, and $N_j < 10$. Print these in. your notebook

(e) Make the following plots:

- $\hat{\Sigma}_j$ vs $R_{j,mid}$ with approximate uncertainty $\sqrt{\frac{N_j}{A_j}}$,
- N_j vs $R_{j,mid}$ with approximate uncertainty $\sqrt{N}$,

(f) For each binning scheme, identify one specific radius range where the finer binning creates "mostly empty" bins. Choose a number of bins for your further analysis based on the numbers you've calculated in (d) and the figures you've made in (e) (does not have to be 15 or 30, it can be a different number). Justify your binning based on what you can see in the figure, and discuss what trade-off you are making in terms of the number of low-count bins versus any potential small-scale structure visible in the figures.

Exercise 2 — Likelihood & Prior (4 pts)

(a) You must decide which likelihood is appropriate for the binned radial dataset you constructed. Typical choices for a likelihood include:

- Poisson likelihood for count data: $N_j \sim Poisson(\Sigma(R_{j,mid})A_j)$.
- Gaussian likelihood : $N_i \sim Normal(\Sigma(R_{j,mid}), \sigma_{\Sigma,j}^2)$ with $\sigma_{\Sigma,j} \approx \sqrt{N_j}$.
- Another reasonable approximation.

In ~3–5 sentences, explicitly connect your choice of likelihood to the number of zero/low-count bins and where they occur in radius. State one *specific* failure mode you would expect if you used the other likelihood, referencing your computed results.

(b) We will now connect the model introduced in the beginning of the assignment, the King radial profile model, to our data, using the likelihood you chose. Write a function that computes your likelihood: you'll want that function that takes in a vector of parameters and the data and computes the *logarithm* of the likelihood you've chosen for a given model. For a set of parameters you guess should be reasonable based on the profile you've plotted in Exercise 1, show that the log-likelihood returns a finite value.

Hints: (1) If you chose a Poisson likelihood, you need to convert the model surface density into estimated counts per bin. A common approximation *is* $\lambda_j \approx \Sigma(R_{j,mid})A_j$ (in real applications, you should be integrating $\Sigma(R_j)$ between boundaries j and $j+1$, but I will spare you that hassle for the purpose of this exercise). (2) Sometimes, the surface density you calculate may be zero for some radii without the explicit background parameter. In the Poisson likelihood, this will cause the log-likelihood to be `-inf`. A reasonably ok hack in our current problem[1] is to add a really small number (e.g. 1e-10) to the King profile, which will ensure that the likelihood doesn't cause an error.

[1] For the purposes of this exercise! In a real analysis, you should include a background parameter!

(c) You will perform posterior inference using Markov Chain Monte Carlo. Implement the following priors for the three parameters in the model:
- r_c ~ Uniform(0, 1) [arcmin]
- r_t ~ Uniform(r_c, 10) [arcmin]
- $\Sigma_0 \sim HalfNormal(\sigma_0)$ with some appropriate scale σ_0

(i) Choose an appropriate scale for Σ_0 and justify your choice using prior results and figures.

(ii) Choose some possible parameter values for each parameter: two values you think are reasonable in the context of your cluster, and two that are way off, and compute the prior log-probability density for them. Is the output of the prior sensible?
Hint: While generally, we don't want to choose priors based on our data, for uninformative or weakly informative priors, it can sometimes be reasonable to choose priors based on the data. For example, many models have an amplitude parameter setting the vertical normalization, which often doesn't have a direct physical meaning, because it will depend on the details of the experiment and its sensitivity. In this case, using e.g. the largest measured surface density to inform the hyperparameters governing your prior can be reasonable.

(iii) Write a function that computes the logarithm of the joint prior probability density given an input vector of parameters.

Exercise 3 – Posterior Inference (4 pts)

(a) Write a function that computes the log-posterior, using the two functions for the log-likelihood and the log-prior.

(b) Choose an MCMC algorithm (a standard choice in astronomy/physics is the affine-invariant ensemble sampler implemented in the `emcee` library). **Note**: you are not required to write your own MCMC sampler! You are free to use any MCMC sampling library you would like. Make an initial choice for the total number of steps and the number to be discarded as burn-in steps, and record those numbers. Also record the number of chains/walkers, and any hyperparameters (i.e. parameters of the sampling process) you may have set (note: not all algorithms require you to manually set parameters).

(c) Compute starting positions: To get MCMC to run well, you'll need to choose reasonable starting positions. Often, you use a maximum likelihood fit, or you can calculate these from the data (e.g. the mean of the last few bins makes a good starting position for the background rate). Here, choose reasonable starting positions based on summaries you can compute from your cluster profile or from what you can eyeball from your observed profile. Justify how you've chosen the starting positions. Then slightly randomize the starting positions for each chain by drawing from a small Gaussian (variance ~10^-2) around those starting positions.
Hint: make sure that after drawing from the Gaussian, your starting points are all within the prior bounds by checking that the logprior of those starting positions is a well-defined number! You might want to generate more starting positions than you need and then simply choose a subset that is guaranteed to be within the prior bounds.

(d) Run MCMC for the model with at least 4 chains (for an equivalent multi-walker set-up, if you use `emcee`, you'll need at least $2 \times N_p$ walkers, where N_p is the number of parameters in the model.).

Exercise 4 – Diagnostics (6 pts)

Compute the following diagnostics for your MCMC results:

(a) **Trace plots**: Show trace plots for all parameters of the entire chain (with burn-in). Clearly mark the part of the chain to be discarded as burn-in.
  (i) Based on the trace plots, choose one of the following and justify using your trace plots:
    i. My MCMC chains are well-converged and sampling from the posterior
    ii. My MCMC chains are not converged in parameters [add the relevant parameters].
  (ii) Did you leave enough steps for burn-in? Justify your answer based on the trace plots. If you chose too few burn-in steps, adjust your numbers and re-run the MCMC chains.

(b) **Integrated autocorrelation time and effective sample size** (after discarding burn-in!): Compute the integrated autocorrelation time τ_{int} for each parameter and report the effective sample size $N_{eff} \approx N/\tau_{int}$. How long would you need to run your chain in order to achieve that sample? Justify your answers using the autocorrelation time.
Hint: Most libraries have implemented a function to calculate the autocorrelation time, you shouldn't need to write code for this yourself.

(c) **Corner plot**: Make a corner plot of the parameters. Comment on the shape of the 1D and 2D posteriors: do they look Gaussian? Are they skewed? Are there multiple peaks for some of the parameters? Are there a lot of outliers from the distribution, or weird features? Are there parameters that are clearly correlated? Are all samples well-contained within the prior range, or are some truncated against the prior bounds? For each (unexpected) feature that you observe, give on one possible reason for what you see, and justify your answer based on any of your previous plots **Hint**: comparing the values of the posterior with the radial profile is a good place to start start!.

(d) **Posterior predictive checks**: Generate replicated datasets under your chosen likelihood using posterior draws. Note: to do this, first generate a model profile using a posterior sample, then use the distribution you've used to define the likelihood to add noise. Make PPC overlay plot: observed N_j, plus at least 20 replicated profiles. Are the replicated profiles similar to what you've observed? Can you see systematic biases (i.e. where the simulated profiles overestimate or underestimate the observed counts)? Does the noise distribution of your simulated data sets look similar to that of the data?

(e) **Compute the posterior median and 68% credible interval for each parameter**
Hint: the function `np.percentile()` makes this reasonably straightforward, as does `arviz.summary()`!

(f) **Interpretation**: Based on the plots you've made above, discuss in a few sentences whether you think the posteriors you derived can be used to draw scientific conclusions about the cluster. Reference specific results/figures and the features you see in these figures in your answer.

Hint: there are a few different libraries that you can use to make these plots. Most can be done with just regular matplotlib or seaborn, , and with the corner plot/pair plot libraries you've explored in assignment 1, but a good option to explore in this context is [arviz](), which is designed to help you explore sampling results. You are not required to write the code to do these yourself from scratch: you are very welcome here to make use of resources that already exist, e.g. in existing packages, on the web, with appropriate attribution. In grading, we will focus on whether your interpretations are closely tied to your observations and make sense, and whether the plots are designed in such a way that they help you (and us) draw conclusions about the validity of your posteriors. Small exception: the posterior predictive checks will require you to write some of your own code to work, because that's not easily generalizable.

Exercise 5 – Priors revisited (6 pts)

(a) Based on the results from the previous part, discuss whether the initial priors are a reasonable choice, what kind of adjustment you might make to improve them, and how you expect that adjustment to affect the diagnostics you computed in the previous section.

(b) Here are three possible choices for the prior for core radius r_c:
- $r_c \sim Uniform(0, 0.5)$
- $r_c \sim LogUniform(-5, \log 2r_t)$
- $r_c \sim Uniform(0, R_{\max})$, where R_{max} is the largest bin size

Rank the three options from most to least appropriate, and justify your rankings in a short paragraph. Your justification must reference the properties of your dataset (e.g. bin widths, R_{max}) and what the parameters represent (which will help you exclude unphysical choices).

(c) For r_t, first check whether your cluster has a Wikipedia[i] page, with a tidal radius and distance given. If so, note the the given tidal radius and its uncertainty (and include the reference page where you obtained the information), then translate both into arcmin using $arcmin = \frac{size}{distance} \times \frac{180}{\pi} \times 60$, and use these values to define a Gaussian prior for r_t (!). If you can't find any values for your cluster, define a reasonably conservative prior based on radial profile and justify your choice using specific observations from previous figures you've made.

(d) Implement your new priors, re-run your MCMC chain and compute the same diagnostics as before.
   (i) Is the MCMC sampler converged? Use the numbers and Figures to justify your answer.
   (ii) Comment on any surprising features in the corner plot that you've made, and name one reason per feature you identify that could explain them.
   (iii) Is the posterior predictive distribution a better approximation to the real data than for the previous prior? What has changed? Are there features in the data that are still not well-explained by the model? Highlight specific features in the Figure to justify your argument.
   (iv) Compute posterior medians and credible intervals for all parameters, and compare these to the posterior medians and credible intervals computed on the MCMC chains using the previous priors. Have any parameter estimates shifted? In which direction? State what change in the prior you think has the largest effect on the results.

---
[i] normally you would find that information in scientific papers, but I won't make you search through papers for this exercise